

MYSTIC WEATHER APP - COMPREHENSIVE EXECUTION PLAN

For AI Collaborators: Explicit Task Breakdown & Deliverables

Project: Mystic Weather Mobile Application

Foundation: NINE65 FHE Cryptographic System + DELUGE Weather Engine

Target Platform: React Native / Expo (iOS + Android)

Status: Planning Phase - Ready for Implementation

Date: January 9, 2026

EXECUTIVE SUMMARY

This document provides **explicit, step-by-step instructions** for building a production-grade mobile weather application that leverages the MYSTIC/DELUGE weather prediction system. The app will display:

- Current Weather Screen** - Real-time atmospheric conditions with chaos signatures
- Forecast Screen** - 7-day forecast with attractor-based flood risk predictions
- Alert System** - Multi-level warnings (CLEAR → WATCH → ADVISORY → WARNING → EMERGENCY)
- Historical Data** - Past events and attractor basin matches

The MYSTIC system provides the **mathematical backbone**: exact integer-based weather prediction using Lorenz attractors, Lyapunov exponents, and zero-drift chaos mathematics.

PART 1: SYSTEM ARCHITECTURE OVERVIEW

1.1 MYSTIC System Components (Existing - DO NOT MODIFY)

Plain Text

```
MYSTIC/DELUGE System Architecture
├─ NINE65 Cryptographic Foundation
│  └─ Exact Integer Arithmetic (src/arithmetic/)
│     └─ K-Elimination (60-year RNS bottleneck solved)
```

```

| | | — Persistent Montgomery (zero conversion overhead)
| | | — FFT-based NTT (26× speedup)
| | | — Exact CT×CT multiplication (zero drift)
| | — Homomorphic Encryption (src/ops/)
| | | — BFV scheme implementation
| | | — Encrypted operations
| | | — Noise budget tracking
| — Quantum Simulation (src/quantum/, src/ahop/)
| | — Grover search (10,000+ iterations)
| | — Entanglement (Bell/GHZ states)
| | — Teleportation (K-channel)
|
| — DELUGE Weather Engine (src/chaos/)
| | — Exact Lorenz Attractor (128-bit integers)
| | | — Zero floating-point drift
| | — Lyapunov Exponent Analysis
| | | — Chaos signature extraction
| | — Attractor Basin Detection
| | | — Flash flood pattern matching
| — Weather State Transformation
| | — Sensor data → Phase space mapping

```

Key Files to Integrate:

- `src/chaos/weather.rs` - FloodDetector, WeatherState, AlertLevel
- `src/chaos/lorenz.rs` - ExactLorenz, LorenzState
- `src/chaos/lyapunov.rs` - LyapunovAnalyzer, ChaosSignature
- `src/chaos/attractor.rs` - AttractorDetector, AttractorId

Do NOT modify these files - they are the mathematical foundation.

1.2 Mobile App Layer (NEW - TO BE BUILT)

Plain Text

Mobile App Architecture

```

| — Frontend Layer (React Native/Expo)
| | — Navigation Stack
| | | — HomeScreen (Current Weather)
| | | — ForecastScreen (7-day Forecast)
| | | — AlertsScreen (Active Warnings)
| | | — HistoryScreen (Past Events)
| | | — SettingsScreen (Preferences)
| | — UI Components
| | | — WeatherCard (Current conditions)
| | | — ForecastCard (Daily forecast)

```

```
| | | └─ AlertBanner (Active warnings)
| | | └─ ChaosSignatureVisualization (Attractor plots)
| | | └─ TimelineChart (Historical data)
| └─ State Management
|   └─ Redux/Context for weather data
└─ Backend API Layer (Node.js/Express)
    └─ REST Endpoints
        ├── GET /api/weather/current
        ├── GET /api/weather/forecast
        ├── GET /api/weather/alerts
        ├── GET /api/weather/history
        └─ POST /api/weather/location
    └─ Data Processing
        ├── Sensor data ingestion
        ├── MYSTIC prediction engine calls
        └─ Alert generation
    └─ Database
        ├── Current weather cache
        ├── Forecast data
        ├── Alert history
        └─ User preferences
└─ MYSTIC Integration Layer
    └─ Rust FFI Bindings
        ├── Call FloodDetector.predict()
        ├── Call LyapunovAnalyzer.step()
        └─ Call AttractorDetector.match()
    └─ Data Transformation
        ├── API sensor data → WeatherState
        ├── WeatherState → Lorenz coordinates
        └─ Predictions → Alert levels
    └─ Caching Strategy
        ├── Cache predictions (1-hour TTL)
        ├── Cache forecasts (6-hour TTL)
        └─ Cache alerts (real-time)
```

PART 2: DETAILED TASK BREAKDOWN

TASK GROUP 1: BACKEND API SETUP

TASK 1.1: Initialize Node.js/Express Backend

Assignee: Backend Developer

Estimated Time: 2-3 hours

Dependencies: None

Deliverables:

- ☐ Create `backend/` directory structure
- ☐ Initialize Node.js project with `package.json`
- ☐ Install dependencies:

Bash

```
npm install express cors dotenv axios
npm install --save-dev nodemon typescript @types/express
```

- ☐ Create `backend/src/server.ts` with Express app
- ☐ Create `.env` template with required variables
- ☐ Set up basic error handling middleware
- ☐ Create health check endpoint: `GET /api/health`

Acceptance Criteria:

- Server starts on `http://localhost:3000`
- Health check returns `{ status: "ok" }`
- CORS configured for mobile app origin
- All dependencies installed without errors

Code Template:

TypeScript

```
// backend/src/server.ts
import express from 'express';
import cors from 'cors';

const app = express( );
app.use(cors());
app.use(express.json());

app.get('/api/health', (req, res) => {
  res.json({ status: 'ok', timestamp: new Date().toISOString() });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
```

```
console.log(`Server running on port ${PORT}`);
});
```

TASK 1.2: Create Weather Data Models & Types

Assignee: Backend Developer

Estimated Time: 1-2 hours

Dependencies: TASK 1.1

Deliverables:

- ☐ Create `backend/src/types/weather.ts` with TypeScript interfaces:

TypeScript

```
interface RawSensorData {
  temp: number; // °C
  dewpoint: number; // °C
  pressure: number; // hPa
  wind_speed: number; // m/s
  wind_dir: number; // degrees
  rain_rate: number; // mm/hr
  soil_moisture: number; // %
  stream_level: number; // cm
  ocean_temp: number; // °C
  wave_height: number; // m
  tide_level: number; // cm
  solar_xray: number; // W/m²
  geomagnetic_kp: number; // Kp index
  solar_wind_speed: number; // m/s
  lunar_phase: number; // 0-1
  tidal_force: number; // 0-1
  cosmic_ray_flux: number;
  seismic_mag: number; // Magnitude
  seismic_dist: number; // km
}

interface WeatherState {
  instability: number; // CAPE-like
  moisture: number; // Moisture flux
  shear: number; // Wind shear
  timestamp: number; // Unix seconds
  station_id: number;
  raw: RawSensorData;
}
```

```
interface AlertLevel {
  level: 'CLEAR' | 'WATCH' | 'ADVISORY' | 'WARNING' | 'EMERGENCY';
  probability: number; // 0.0 - 1.0
  description: string;
}

interface FloodPrediction {
  probability: number;
  alert: AlertLevel;
  time_to_onset?: number; // hours
  attractor?: string;
  confidence: number;
  signature: ChaosSignature;
  action: string;
}

interface ChaosSignature {
  lyapunov_exponent: number;
  dimension: number;
  entropy: number;
  timestamp: number;
}

interface CurrentWeatherResponse {
  location: {
    name: string;
    lat: number;
    lon: number;
    timezone: string;
  };
  current: {
    temp: number;
    feels_like: number;
    humidity: number;
    pressure: number;
    wind_speed: number;
    wind_direction: number;
    rainfall: number;
    condition: string;
    icon: string;
    timestamp: number;
  };
  mystic: {
    prediction: FloodPrediction;
    chaos_signature: ChaosSignature;
    attractor_match?: string;
  };
}
```

```

interface ForecastResponse {
  location: {
    name: string;
    lat: number;
    lon: number;
  };
  forecast: ForecastDay[];
}

interface ForecastDay {
  date: string; // YYYY-MM-DD
  high_temp: number;
  low_temp: number;
  condition: string;
  icon: string;
  rainfall_probability: number;
  flood_risk: {
    level: string;
    probability: number;
    confidence: number;
  };
  chaos_signature: ChaosSignature;
}

```

- ☐ Create `backend/src/types/index.ts` exporting all types
- ☐ Create `backend/src/constants/alerts.ts` with alert level definitions

Acceptance Criteria:

- All types compile without errors
- Types match MYSTIC system outputs
- Types are properly exported and importable

TASK 1.3: Integrate MYSTIC Rust Library via FFI

Assignee: Rust/Backend Developer

Estimated Time: 3-4 hours

Dependencies: TASK 1.1, MYSTIC system available

Deliverables:

- ☐ Create `backend/src/mystic/bindings.ts` - TypeScript FFI wrapper
- ☐ Create `backend/src/mystic/weather-engine.ts` - High-level API

- ☐ Build Rust library as WASM or native module
- ☐ Create initialization function:

TypeScript

```
// backend/src/mystic/weather-engine.ts
import { FloodDetector, WeatherState } from './bindings';

export class MysticWeatherEngine {
  private detector: FloodDetector;

  constructor() {
    this.detector = new FloodDetector();
  }

  predict(sensorData: RawSensorData): FloodPrediction {
    const state = WeatherState.fromSensors(
      sensorData,
      Date.now() / 1000
    );
    return this.detector.predict(state);
  }

  updateWithNewReading(sensorData: RawSensorData): void {
    const state = WeatherState.fromSensors(
      sensorData,
      Date.now() / 1000
    );
    this.detector.update(state);
  }
}
```

- ☐ Create unit tests for FFI wrapper
- ☐ Document expected input/output formats
- ☐ Create error handling for Rust panics

Acceptance Criteria:

- Rust library successfully compiles
- FFI bindings work without crashes
- Can call FloodDetector.predict() from Node.js
- All Rust errors properly caught and converted to JS errors

Technical Notes:

- Use `napi-rs` for Node.js native bindings OR
 - Use `wasm-pack` for WebAssembly compilation
 - Ensure thread safety for concurrent predictions
-

TASK 1.4: Create Weather Data API Endpoints

Assignee: Backend Developer

Estimated Time: 2-3 hours

Dependencies: TASK 1.2, TASK 1.3

Deliverables:

- ☐ Implement `GET /api/weather/current` endpoint

TypeScript

```
// Returns CurrentWeatherResponse
// Input: ?lat=29.4&lon=-98.5&station_id=1
// Calls MysticWeatherEngine.predict()
// Returns current conditions + MYSTIC prediction
```

- ☐ Implement `GET /api/weather/forecast` endpoint

TypeScript

```
// Returns ForecastResponse
// Input: ?lat=29.4&lon=-98.5&days=7
// Generates 7-day forecast with flood risk
```

- ☐ Implement `GET /api/weather/alerts` endpoint

TypeScript

```
// Returns array of active alerts
// Input: ?lat=29.4&lon=-98.5&radius_km=50
// Filters alerts by location and severity
```

- ☐ Implement `GET /api/weather/history` endpoint

TypeScript

```
// Returns historical events
// Input: ?lat=29.4&lon=-98.5&days=30
```

```
// Returns past predictions and actual outcomes
```

☐ Implement `POST /api/weather/location` endpoint

TypeScript

```
// Saves user's favorite locations
// Input: { name, lat, lon }
// Returns saved location with ID
```

Acceptance Criteria:

- All endpoints return proper HTTP status codes
- Responses match TypeScript interfaces
- Error handling for invalid inputs
- Rate limiting configured (100 req/min per IP)
- CORS headers properly set

TASK 1.5: Create Data Caching Layer

Assignee: Backend Developer

Estimated Time: 2 hours

Dependencies: TASK 1.4

Deliverables:

☐ Install Redis or use in-memory cache:

Bash

```
npm install redis
```

☐ Create `backend/src/cache/weather-cache.ts` :

TypeScript

```
export class WeatherCache {
  private cache: Map<string, CachedWeather>;
  private ttl = {
    current: 5 * 60 * 1000,    // 5 minutes
    forecast: 6 * 60 * 60 * 1000, // 6 hours
    alerts: 1 * 60 * 1000      // 1 minute
  };
};
```

```

    get(key: string): any | null {
      const entry = this.cache.get(key);
      if (!entry) return null;
      if (Date.now() > entry.expiresAt) {
        this.cache.delete(key);
        return null;
      }
      return entry.data;
    }

    set(key: string, data: any, ttlMs: number): void {
      this.cache.set(key, {
        data,
        expiresAt: Date.now() + ttlMs
      });
    }
  }
}

```

- ☐ Integrate cache into API endpoints
- ☐ Add cache invalidation on sensor updates
- ☐ Create cache statistics endpoint: GET /api/cache/stats

Acceptance Criteria:

- Cache hits reduce response time by 50%+
- Cache properly expires old entries
- Cache can be manually cleared via admin endpoint

TASK GROUP 2: MOBILE APP FRONTEND

TASK 2.1: Initialize React Native/Expo Project

Assignee: Mobile Developer

Estimated Time: 1-2 hours

Dependencies: None

Deliverables:

- ☐ Create React Native project:

Bash

```
npx create-expo-app mystic-weather
cd mystic-weather
npm install expo-router expo-splash-screen expo-font
npm install @react-navigation/native @react-navigation/bottom-tabs
npm install axios redux @reduxjs/toolkit
npm install react-native-svg react-native-chart-kit
```

☐ Set up project structure:

Plain Text

```
mystic-weather/
├── app/
│   ├── _layout.tsx      # Root layout
│   ├── index.tsx        # Home screen
│   ├── forecast.tsx     # Forecast screen
│   ├── alerts.tsx       # Alerts screen
│   ├── history.tsx      # History screen
│   └── settings.tsx     # Settings screen
├── src/
│   ├── components/      # Reusable components
│   ├── screens/         # Screen components
│   ├── services/        # API services
│   ├── store/           # Redux store
│   ├── types/           # TypeScript types
│   └── utils/           # Utilities
├── assets/
│   ├── icons/
│   ├── images/
│   └── fonts/
└── app.json
```

- ☐ Configure `app.json` with app metadata
- ☐ Set up TypeScript configuration
- ☐ Create `.env` file for API endpoint

Acceptance Criteria:

- App runs on iOS simulator/Android emulator
- Navigation structure works
- All dependencies installed without conflicts

TASK 2.2: Create API Service Layer

Assignee: Mobile Developer

Estimated Time: 1-2 hours

Dependencies: TASK 2.1

Deliverables:

☐ Create `src/services/weather-api.ts` :

TypeScript

```
import axios from 'axios';
import {
  CurrentWeatherResponse,
  ForecastResponse,
  AlertLevel
} from '../types/weather';

const API_BASE = process.env.EXPO_PUBLIC_API_URL ||
'http://localhost:3000';

export const weatherAPI = {
  getCurrentWeather: async (lat: number, lon: number ) => {
    const response = await axios.get<CurrentWeatherResponse>(
      `${API_BASE}/api/weather/current`,
      { params: { lat, lon } }
    );
    return response.data;
  },

  getForecast: async (lat: number, lon: number, days: number = 7) => {
    const response = await axios.get<ForecastResponse>(
      `${API_BASE}/api/weather/forecast`,
      { params: { lat, lon, days } }
    );
    return response.data;
  },

  getAlerts: async (lat: number, lon: number, radiusKm: number = 50) => {
    const response = await axios.get<AlertLevel[]>(
      `${API_BASE}/api/weather/alerts`,
      { params: { lat, lon, radius_km: radiusKm } }
    );
    return response.data;
  },

  getHistory: async (lat: number, lon: number, days: number = 30) => {
    const response = await axios.get(
      `${API_BASE}/api/weather/history`,
```

```
        { params: { lat, lon, days } }
    );
    return response.data;
}
};
```

- ☐ Create error handling wrapper
- ☐ Add request/response logging
- ☐ Create retry logic for failed requests

Acceptance Criteria:

- API calls work with mock data
- Errors properly caught and logged
- Requests include proper headers

TASK 2.3: Create Redux Store for Weather Data

Assignee: Mobile Developer

Estimated Time: 1-2 hours

Dependencies: TASK 2.2

Deliverables:

- ☐ Create `src/store/weatherSlice.ts` :

TypeScript

```
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import { weatherAPI } from '../services/weather-api';

export const fetchCurrentWeather = createAsyncThunk(
  'weather/fetchCurrent',
  async ({ lat, lon }: { lat: number; lon: number }) => {
    return await weatherAPI.getCurrentWeather(lat, lon);
  }
);

export const fetchForecast = createAsyncThunk(
  'weather/fetchForecast',
  async ({ lat, lon, days }: { lat: number; lon: number; days: number })
=> {
  return await weatherAPI.getForecast(lat, lon, days);
}
);
```

```

const weatherSlice = createSlice({
  name: 'weather',
  initialState: {
    current: null,
    forecast: null,
    alerts: [],
    loading: false,
    error: null
  },
  extraReducers: (builder) => {
    builder
      .addCase(fetchCurrentWeather.pending, (state) => {
        state.loading = true;
      })
      .addCase(fetchCurrentWeather.fulfilled, (state, action) => {
        state.current = action.payload;
        state.loading = false;
      })
      .addCase(fetchCurrentWeather.rejected, (state, action) => {
        state.error = action.error.message;
        state.loading = false;
      });
  }
});

export default weatherSlice.reducer;

```

- ☐ Create `src/store/index.ts` with store configuration
- ☐ Create location slice for user locations
- ☐ Create settings slice for preferences

Acceptance Criteria:

- Redux store initializes without errors
- Actions dispatch successfully
- State updates properly

TASK 2.4: Build Current Weather Screen

Assignee: Mobile Developer

Estimated Time: 3-4 hours

Dependencies: TASK 2.3

Deliverables:

- ☐ Create `app/index.tsx` (Home/Current Weather Screen):

TypeScript

```
import React, { useEffect } from 'react';
import { View, Text, StyleSheet, ScrollView } from 'react-native';
import { useDispatch, useSelector } from 'react-redux';
import { fetchCurrentWeather } from '../src/store/weatherSlice';

export default function HomeScreen() {
  const dispatch = useDispatch();
  const { current, loading } = useSelector((state) => state.weather);

  useEffect(() => {
    // Get user's location (use geolocation API)
    // Then fetch weather
    dispatch(fetchCurrentWeather({ lat: 29.4, lon: -98.5 }));
  }, []);

  if (loading) return <Text>Loading...</Text>;
  if (!current) return <Text>No data</Text>;

  return (
    <ScrollView style={styles.container}>
      {/* Location header */}
      <View style={styles.header}>
        <Text style={styles.locationName}>{current.location.name}</Text>
        <Text style={styles.timestamp}>
          {new Date(current.current.timestamp *
1000).toLocaleTimeString()}
        </Text>
      </View>

      {/* Current conditions */}
      <View style={styles.currentCard}>
        <Text style={styles.temperature}>{current.current.temp}°C</Text>
        <Text style={styles.condition}>{current.current.condition}</Text>
        <Text style={styles.feelsLike}>
          Feels like {current.current.feels_like}°C
        </Text>
      </View>

      {/* Detailed metrics */}
      <View style={styles.metricsGrid}>
        <MetricCard
          label="Humidity"
```



```

        value={`$${current.current.humidity}%`}
      />
    <MetricCard
      label="Pressure"
      value={`$${current.current.pressure} hPa`}
    />
    <MetricCard
      label="Wind"
      value={`$${current.current.wind_speed} m/s`}
    />
    <MetricCard
      label="Rainfall"
      value={`$${current.current.rainfall} mm/hr`}
    />
  </View>

  {/* MYSTIC Prediction */}
  <View style={styles.mysticCard}>
    <Text style={styles.mysticTitle}>MYSTIC Prediction</Text>
    <AlertBanner alert={current.mystic.prediction.alert} />
    <Text style={styles.probability}>
      Flood Probability: {(current.mystic.prediction.probability *
100).toFixed(1)}%
    </Text>
    {current.mystic.prediction.time_to_onset && (
      <Text style={styles.timeToOnset}>
        Estimated onset: {current.mystic.prediction.time_to_onset}
hours
      </Text>
    )}
    <ChaosSignatureChart signature={current.mystic.chaos_signature}
  />
  </View>
</ScrollView>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f5f5f5' },
  header: { padding: 20, backgroundColor: '#fff' },
  locationName: { fontSize: 28, fontWeight: 'bold' },
  timestamp: { fontSize: 12, color: '#666', marginTop: 4 },
  currentCard: {
    margin: 16,
    padding: 20,
    backgroundColor: '#fff',
    borderRadius: 12,
    alignItems: 'center'
  }
});

```

```

    },
    temperature: { fontSize: 48, fontWeight: 'bold', color: '#1e88e5' },
    condition: { fontSize: 18, color: '#666', marginTop: 8 },
    feelsLike: { fontSize: 14, color: '#999', marginTop: 4 },
    metricsGrid: {
      flexDirection: 'row',
      flexWrap: 'wrap',
      padding: 8,
      justifyContent: 'space-between'
    },
  },
  mysticCard: {
    margin: 16,
    padding: 20,
    backgroundColor: '#fff',
    borderRadius: 12,
    borderLeftWidth: 4,
    borderLeftColor: '#d32f2f'
  },
  mysticTitle: { fontSize: 16, fontWeight: 'bold', marginBottom: 12 },
  probability: { fontSize: 14, marginTop: 8, color: '#333' },
  timeToOnset: { fontSize: 12, color: '#d32f2f', marginTop: 4,
fontWeight: '600' }
});

```

- ☐ Create `src/components/AlertBanner.tsx` - Alert level display
- ☐ Create `src/components/MetricCard.tsx` - Individual metric display
- ☐ Create `src/components/ChaosSignatureChart.tsx` - Visualization of Lyapunov exponent
- ☐ Add location detection using `expo-location`
- ☐ Add pull-to-refresh functionality

Acceptance Criteria:

- Screen displays current weather correctly
- MYSTIC prediction displayed prominently
- Alert level color-coded (CLEAR=green, WARNING=red, etc.)
- Refreshes data on pull-to-refresh

TASK 2.5: Build Forecast Screen

Assignee: Mobile Developer

Estimated Time: 2-3 hours

Dependencies: TASK 2.3

Deliverables:

- ☐ Create `app/forecast.tsx` :

TypeScript

```
import React, { useEffect } from 'react';
import { View, Text, FlatList, StyleSheet } from 'react-native';
import { useDispatch, useSelector } from 'react-redux';
import { fetchForecast } from '../src/store/weatherSlice';

export default function ForecastScreen() {
  const dispatch = useDispatch();
  const { forecast, loading } = useSelector((state) => state.weather);

  useEffect(() => {
    dispatch(fetchForecast({ lat: 29.4, lon: -98.5, days: 7 }));
  }, []);

  const renderDay = ({ item }) => (
    <View style={styles.dayCard}>
      <Text style={styles.date}>{item.date}</Text>
      <Text style={styles.condition}>{item.condition}</Text>
      <View style={styles.temps}>
        <Text style={styles.tempHigh}>{item.high_temp}°</Text>
        <Text style={styles.tempLow}>{item.low_temp}°</Text>
      </View>
      <View style={styles.floodRisk}>
        <Text style={styles.riskLabel}>Flood Risk:</Text>
        <Text style={[
          styles.riskLevel,
          { color: getRiskColor(item.flood_risk.level) }
        ]}>
          {item.flood_risk.level}
        </Text>
        <Text style={styles.riskProb}>
          {(item.flood_risk.probability * 100).toFixed(0)}%
        </Text>
      </View>
    </View>
  );

  return (
    <FlatList
      data={forecast?.forecast}
      renderItem={renderDay}
      keyExtractor={(item) => item.date}
      contentContainerStyle={styles.container}>
    </FlatList>
  );
}
```

```

    />
  );
}

const getRiskColor = (level: string) => {
  const colors = {
    'CLEAR': '#4caf50',
    'WATCH': '#ff9800',
    'ADVISORY': '#ff6f00',
    'WARNING': '#d32f2f',
    'EMERGENCY': '#b71c1c'
  };
  return colors[level] || '#999';
};

const styles = StyleSheet.create({
  container: { padding: 12 },
  dayCard: {
    backgroundColor: '#fff',
    borderRadius: 8,
    padding: 16,
    marginBottom: 12
  },
  date: { fontSize: 16, fontWeight: 'bold', marginBottom: 8 },
  condition: { fontSize: 14, color: '#666', marginBottom: 8 },
  temps: { flexDirection: 'row', marginBottom: 12 },
  tempHigh: { fontSize: 18, fontWeight: 'bold', marginRight: 16 },
  tempLow: { fontSize: 14, color: '#999' },
  floodRisk: { borderTopWidth: 1, borderTopColor: '#eee', paddingTop: 12
},
  riskLabel: { fontSize: 12, color: '#666', marginBottom: 4 },
  riskLevel: { fontSize: 14, fontWeight: 'bold' },
  riskProb: { fontSize: 12, color: '#999', marginTop: 2 }
});

```

- ☐ Create `src/components/ForecastDayCard.tsx` - Individual day forecast
- ☐ Add chart visualization for temperature trend
- ☐ Add rainfall probability visualization

Acceptance Criteria:

- 7-day forecast displays correctly
 - Flood risk clearly indicated with color coding
 - Smooth scrolling through forecast days
-

TASK 2.6: Build Alerts Screen

Assignee: Mobile Developer

Estimated Time: 2 hours

Dependencies: TASK 2.3

Deliverables:

- ☐ Create `app/alerts.tsx` - Active alerts display
- ☐ Create `src/components/AlertCard.tsx` - Individual alert display
- ☐ Implement alert filtering by severity
- ☐ Add alert details modal
- ☐ Implement alert dismissal

Acceptance Criteria:

- Active alerts displayed with severity color coding
 - Alerts can be dismissed
 - Alerts sorted by severity (highest first)
-

TASK 2.7: Build History Screen

Assignee: Mobile Developer

Estimated Time: 2 hours

Dependencies: TASK 2.3

Deliverables:

- ☐ Create `app/history.tsx` - Historical events display
- ☐ Create timeline visualization of past events
- ☐ Show prediction accuracy metrics
- ☐ Display attractor basin matches

Acceptance Criteria:

- Historical events displayed in chronological order
 - Prediction accuracy shown
 - Attractor matches highlighted
-

TASK 2.8: Build Settings Screen

Assignee: Mobile Developer

Estimated Time: 1-2 hours

Dependencies: TASK 2.3

Deliverables:

- ☐ Create `app/settings.tsx`
- ☐ Location management (add/remove favorites)
- ☐ Notification preferences
- ☐ Alert threshold settings
- ☐ Data refresh interval settings

Acceptance Criteria:

- Settings persist across app restarts
- Location changes update all screens
- Notification preferences respected

TASK GROUP 3: VISUALIZATION & UI COMPONENTS

TASK 3.1: Create Chaos Signature Visualization

Assignee: Mobile Developer

Estimated Time: 2-3 hours

Dependencies: TASK 2.4

Deliverables:

- ☐ Create `src/components/ChaosSignatureChart.tsx`
- ☐ Visualize Lyapunov exponent over time
- ☐ Show attractor basin proximity
- ☐ Display entropy metrics
- ☐ Use `react-native-chart-kit` or `react-native-svg` for rendering

Technical Details:

TypeScript

```
// ChaosSignatureChart should display:  
// 1. Lyapunov exponent trend (line chart)
```

```
// 2. Attractor distance metric (gauge)
// 3. Entropy level (bar)
// 4. Chaos dimension (text)
```

Acceptance Criteria:

- Chart renders without crashing
 - Updates in real-time as new data arrives
 - Responsive to screen size changes
-

TASK 3.2: Create Alert Banner Component

Assignee: Mobile Developer

Estimated Time: 1 hour

Dependencies: TASK 2.4

Deliverables:

- ☐ Create `src/components/AlertBanner.tsx`
- ☐ Color-coded by alert level:
 - CLEAR: Green (#4caf50)
 - WATCH: Yellow (#ff9800)
 - ADVISORY: Orange (#ff6f00)
 - WARNING: Red (#d32f2f)
 - EMERGENCY: Dark Red (#b71c1c)
- ☐ Display alert text and icon
- ☐ Animated pulse for WARNING/EMERGENCY levels

Acceptance Criteria:

- Alert levels display with correct colors
 - Animations smooth and performant
 - Text clearly readable on all backgrounds
-

TASK 3.3: Create Weather Icon System

Assignee: Mobile Developer / Designer

Estimated Time: 1-2 hours

Dependencies: TASK 2.4

Deliverables:

- ☐ Create icon set for weather conditions:
 - Clear/Sunny
 - Cloudy
 - Rainy
 - Stormy
 - Snowy
 - Foggy
 - Windy
- ☐ Create icon set for alert levels
- ☐ Create icon set for metrics (humidity, pressure, wind)
- ☐ Store in `assets/icons/`
- ☐ Create `src/utils/icon-mapper.ts` to map conditions to icons

Acceptance Criteria:

- All weather conditions have corresponding icons
 - Icons are SVG or high-resolution PNG
 - Icons render clearly at all sizes
-

TASK GROUP 4: TESTING & QUALITY ASSURANCE

TASK 4.1: Create Unit Tests for API Service

Assignee: QA/Backend Developer

Estimated Time: 2 hours

Dependencies: TASK 1.4

Deliverables:

- ☐ Install testing framework:

```
Bash

npm install --save-dev jest @testing-library/react-native
```
- ☐ Create `backend/src/__tests__/weather-api.test.ts`

- ☐ Test all API endpoints with mock data
- ☐ Test error handling
- ☐ Test caching behavior

Acceptance Criteria:

- All tests pass
 - 80% code coverage for API layer
 - Error cases properly handled
-

TASK 4.2: Create Integration Tests

Assignee: QA/Mobile Developer

Estimated Time: 2-3 hours

Dependencies: TASK 2.4, TASK 2.5

Deliverables:

- ☐ Test complete user flow:
 1. App launches
 2. Requests current location
 3. Fetches weather data
 4. Displays current weather screen
 5. User navigates to forecast
 6. Forecast displays correctly
 7. User navigates to alerts
 8. Alerts display correctly
- ☐ Test error scenarios:
 - Network unavailable
 - Invalid location
 - API timeout
 - Corrupted data

Acceptance Criteria:

- All user flows work end-to-end
- Error handling graceful

- No crashes or hangs
-

TASK 4.3: Create Performance Tests

Assignee: QA Developer

Estimated Time: 1-2 hours

Dependencies: TASK 2.4, TASK 2.5

Deliverables:

- ☐ Test API response times:
 - Current weather: <500ms
 - Forecast: <1000ms
 - Alerts: <500ms
- ☐ Test UI rendering performance:
 - Home screen: >60 FPS
 - Forecast list: >60 FPS
 - Smooth scrolling
- ☐ Test memory usage:
 - App idle: <100MB
 - Displaying data: <150MB
 - No memory leaks

Acceptance Criteria:

- All performance targets met
 - No memory leaks detected
 - Smooth user experience
-

TASK GROUP 5: DEPLOYMENT & DOCUMENTATION

TASK 5.1: Create Deployment Configuration

Assignee: DevOps/Backend Developer

Estimated Time: 2 hours

Dependencies: TASK 1.5

Deliverables:

- ☐ Create Docker configuration for backend:

Plain Text

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

- ☐ Create `.env.example` with all required variables
- ☐ Create deployment guide for AWS/GCP/DigitalOcean
- ☐ Set up CI/CD pipeline (GitHub Actions)

Acceptance Criteria:

- Docker image builds successfully
- Container runs without errors
- Environment variables properly configured

TASK 5.2: Create API Documentation

Assignee: Backend Developer

Estimated Time: 1-2 hours

Dependencies: TASK 1.4

Deliverables:

- ☐ Create OpenAPI/Swagger documentation
- ☐ Document all endpoints with:
 - Request parameters
 - Response schema
 - Example requests/responses
 - Error codes
 - Rate limits
- ☐ Create Postman collection for testing
- ☐ Host documentation at `/api/docs`

Acceptance Criteria:

- All endpoints documented
 - Examples are accurate
 - Documentation is accessible
-

TASK 5.3: Create User Documentation

Assignee: Technical Writer

Estimated Time: 2-3 hours

Dependencies: All tasks

Deliverables:

- ☐ Create user guide with screenshots
- ☐ Document alert levels and meanings
- ☐ Explain MYSTIC predictions
- ☐ Create troubleshooting guide
- ☐ Create FAQ document

Acceptance Criteria:

- Documentation is clear and comprehensive
 - Screenshots are current
 - All features explained
-

TASK 5.4: Create Developer Documentation

Assignee: Technical Writer / Backend Developer

Estimated Time: 2-3 hours

Dependencies: All tasks

Deliverables:

- ☐ Create architecture documentation
- ☐ Document MYSTIC integration
- ☐ Create setup guide for developers
- ☐ Document build/test procedures
- ☐ Create contribution guidelines

Acceptance Criteria:

- New developers can set up project in <1 hour
 - Architecture clearly explained
 - All code patterns documented
-

PART 3: IMPLEMENTATION TIMELINE

Phase 1: Backend Foundation (Days 1-2)

- TASK 1.1: Express setup
- TASK 1.2: Data models
- TASK 1.3: MYSTIC integration
- TASK 1.4: API endpoints
- TASK 1.5: Caching

Phase 2: Mobile Frontend (Days 3-5)

- TASK 2.1: Expo project
- TASK 2.2: API service
- TASK 2.3: Redux store
- TASK 2.4: Current weather screen
- TASK 2.5: Forecast screen

Phase 3: Additional Screens (Days 6-7)

- TASK 2.6: Alerts screen
- TASK 2.7: History screen
- TASK 2.8: Settings screen

Phase 4: Visualization (Days 8-9)

- TASK 3.1: Chaos visualization
- TASK 3.2: Alert banner
- TASK 3.3: Weather icons

Phase 5: Testing & QA (Days 10-11)

- TASK 4.1: Unit tests
- TASK 4.2: Integration tests
- TASK 4.3: Performance tests

Phase 6: Deployment (Days 12-13)

- TASK 5.1: Docker/deployment
 - TASK 5.2: API documentation
 - TASK 5.3: User documentation
 - TASK 5.4: Developer documentation
-

PART 4: TECHNICAL SPECIFICATIONS

4.1 Backend Stack

- **Runtime:** Node.js 18+
- **Framework:** Express.js
- **Language:** TypeScript
- **Database:** PostgreSQL (for historical data)
- **Cache:** Redis (optional, can use in-memory)
- **MYSTIC Integration:** Rust FFI via NAPI or WASM

4.2 Mobile Stack

- **Framework:** React Native (Expo)
- **Language:** TypeScript
- **State Management:** Redux Toolkit
- **Navigation:** Expo Router
- **HTTP Client:** Axios
- **Charts:** react-native-chart-kit or react-native-svg

4.3 Data Flow

Plain Text

MOBILE APP (React Native)

- └ User opens app
- └ Requests current location (GPS)
- └ Dispatches Redux action: `fetchCurrentWeather(lat, lon)`

HTTP GET `/api/weather/current`

BACKEND API (Express/Node.js)

- └ Receives request with lat/lon
- └ Checks cache for recent data
- └ If cache miss:
 - └ Fetches sensor data from data source
 - └ Transforms to `RawSensorData`
 - └ Calls `MysticWeatherEngine.predict()`
- └ Returns `CurrentWeatherResponse`

MYSTIC WEATHER ENGINE (Rust)

- └ Receives `RawSensorData`
- └ Creates `WeatherState` from sensors
- └ Converts to `LorenzState` (phase space)
- └ Runs `FloodDetector.predict()`:
 - └ Analyzes Lyapunov exponent
 - └ Detects attractor basin match
 - └ Calculates flood probability
 - └ Returns `FloodPrediction`
- └ Returns to backend

BACKEND API (continued)

- └ Receives `FloodPrediction` from MYSTIC
- └ Combines with current weather data
- └ Caches result (5-min TTL)
- └ Returns JSON response to mobile app

JSON response

MOBILE APP (continued)

- └ Receives response
- └ Dispatches Redux action: `setCurrentWeather(data)`
- └ Re-renders `HomeScreen` with:
 - └ Current temperature & conditions

			└ MYSTIC prediction & alert level	
			└ Chaos signature visualization	
			└ Detailed metrics	
			└ User sees complete weather picture	

4.4 Database Schema

SQL

```
-- Users
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  email VARCHAR(255) UNIQUE,
  created_at TIMESTAMP DEFAULT NOW()
);

-- User Locations
CREATE TABLE user_locations (
  id SERIAL PRIMARY KEY,
  user_id INTEGER REFERENCES users(id),
  name VARCHAR(255),
  latitude DECIMAL(10, 6),
  longitude DECIMAL(10, 6),
  is_favorite BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Weather Cache
CREATE TABLE weather_cache (
  id SERIAL PRIMARY KEY,
  location_id INTEGER REFERENCES user_locations(id),
  data JSONB,
  cached_at TIMESTAMP DEFAULT NOW(),
  expires_at TIMESTAMP
);

-- Historical Predictions
CREATE TABLE predictions (
  id SERIAL PRIMARY KEY,
  location_id INTEGER REFERENCES user_locations(id),
  predicted_at TIMESTAMP,
  flood_probability DECIMAL(5, 4),
  alert_level VARCHAR(20),
  actual_outcome VARCHAR(20),
  accuracy DECIMAL(5, 4),
  created_at TIMESTAMP DEFAULT NOW()
```



```
);

-- Alerts
CREATE TABLE alerts (
  id SERIAL PRIMARY KEY,
  location_id INTEGER REFERENCES user_locations(id),
  alert_level VARCHAR(20),
  message TEXT,
  issued_at TIMESTAMP,
  expires_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW()
);
```

PART 5: CRITICAL SUCCESS FACTORS

5.1 MYSTIC Integration

- **Critical:** FFI bindings must be rock-solid
- **Test:** Run MYSTIC demo independently before integration
- **Fallback:** Have mock predictions ready if WASM compilation fails
- **Performance:** Predictions must complete in <100ms

5.2 Real-Time Updates

- **Critical:** Alerts must be delivered immediately
- **Implementation:** Use WebSockets or Server-Sent Events for real-time
- **Fallback:** Poll every 5 minutes if real-time unavailable

5.3 Data Accuracy

- **Critical:** Predictions must match MYSTIC system outputs exactly
- **Validation:** Compare backend predictions with Rust binary output
- **Testing:** Use known test cases from MYSTIC documentation

5.4 User Experience

- **Critical:** App must feel responsive and fast
 - **Target:** <500ms for all API calls
 - **Optimization:** Aggressive caching, background refresh
-

PART 6: RISK MITIGATION

Risk	Probability	Impact	Mitigation
MYSTIC FFI compilation fails	Medium	High	Pre-test WASM build, have mock ready
API response too slow	Medium	High	Implement aggressive caching, optimize queries
Mobile app crashes on startup	Low	High	Comprehensive error handling, extensive testing
Data accuracy issues	Low	High	Validate against known test cases
Network unavailable	Medium	Medium	Offline mode with cached data
Memory leaks in React Native	Low	Medium	Regular profiling, React.memo usage

PART 7: SUCCESS METRICS

7.1 Performance Metrics

- API response time: <500ms (95th percentile)
- App startup time: <3 seconds
- Memory usage: <150MB
- Frame rate: >60 FPS

7.2 Reliability Metrics

- API uptime: >99.5%
- Prediction accuracy: >85% (match MYSTIC system)
- Error rate: <0.1%
- Crash rate: <0.01%

7.3 User Metrics

- User retention: >70% (7-day)
 - Daily active users: >50% of installs
 - Average session length: >5 minutes
 - Feature adoption: >80% use alerts
-

PART 8: DELIVERABLES CHECKLIST

Backend

- ☐ Express server running on port 3000
- ☐ All API endpoints implemented and tested
- ☐ MYSTIC integration working
- ☐ Caching layer functional
- ☐ Database schema created
- ☐ Docker image builds
- ☐ API documentation complete

Mobile App

- ☐ Expo project initialized
- ☐ All 5 screens implemented
- ☐ Redux store working
- ☐ API integration complete
- ☐ All components rendering
- ☐ Navigation working
- ☐ Settings persisting

Testing

- ☐ Unit tests passing (>80% coverage)
- ☐ Integration tests passing
- ☐ Performance tests passing

☐ Manual testing completed

Documentation

- ☐ API documentation
- ☐ User guide
- ☐ Developer guide
- ☐ Deployment guide
- ☐ Architecture documentation

PART 9: CONTACT & ESCALATION

Technical Questions

- **MYSTIC System:** Refer to `/home/ubuntu/mystic_system/nine65_v2_complete/README.md`
- **FFI Integration:** Consult NAPI-RS or WASM-Pack documentation
- **React Native:** Refer to Expo documentation

Blockers

1. **MYSTIC compilation fails:** Escalate to Rust expert
2. **FFI binding issues:** Consult NAPI-RS community
3. **Performance issues:** Profile with DevTools, optimize hotspots
4. **Data accuracy:** Compare with MYSTIC demo output

APPENDIX A: MYSTIC SYSTEM FILES REFERENCE

File	Purpose	Integration Point
src/chaos/weather.rs	FloodDetector, WeatherState	Call FloodDetector.predict()
src/chaos/lorenz.rs	Exact Lorenz attractor	Used by FloodDetector internally
src/chaos/lyapunov.rs	Lyapunov analysis	Used by FloodDetector internally

<code>src/chaos/attractor.rs</code>	Attractor basin detection	Used by FloodDetector internally
<code>src/bin/mystic_demo.rs</code>	Reference implementation	Test against this output

APPENDIX B: ENVIRONMENT VARIABLES

Backend (.env)

Plain Text

```
NODE_ENV=production
PORT=3000
DATABASE_URL=postgresql://user:pass@localhost/mystic
REDIS_URL=redis://localhost:6379
MYSTIC_DATA_SOURCE=api # or 'synthetic' for testing
NOAA_CDO_TOKEN=your_token_here
NWS_USER_AGENT=MYSTIC/1.0 (contact: you@example.com)
```

Mobile (.env)

Plain Text

```
EXPO_PUBLIC_API_URL=http://localhost:3000
EXPO_PUBLIC_APP_NAME=Mystic Weather
```

APPENDIX C: TESTING CHECKLIST

Before Deployment

- ☐ All unit tests passing
- ☐ All integration tests passing
- ☐ Performance benchmarks met
- ☐ Security audit completed
- ☐ API documentation reviewed
- ☐ User documentation reviewed

- ☐ Deployment guide tested
 - ☐ Rollback procedure documented
-

Document Version: 1.0

Last Updated: January 9, 2026

Status: Ready for Implementation

Next Step: Assign tasks to AI collaborators and begin Phase 1 (Backend Foundation)